



Chapitre 2: **Tas (Heap)**

Chapitre2: Tas (Heap)

Définitions

Définition:

Un **arbre binaire partiellement ordonné** est un arbre dont chaque nœud est étiqueté par une valeur telle que la valeur d'un nœud est toujours inférieure ou égale à la valeur de chacun de ses fils.

Définition:

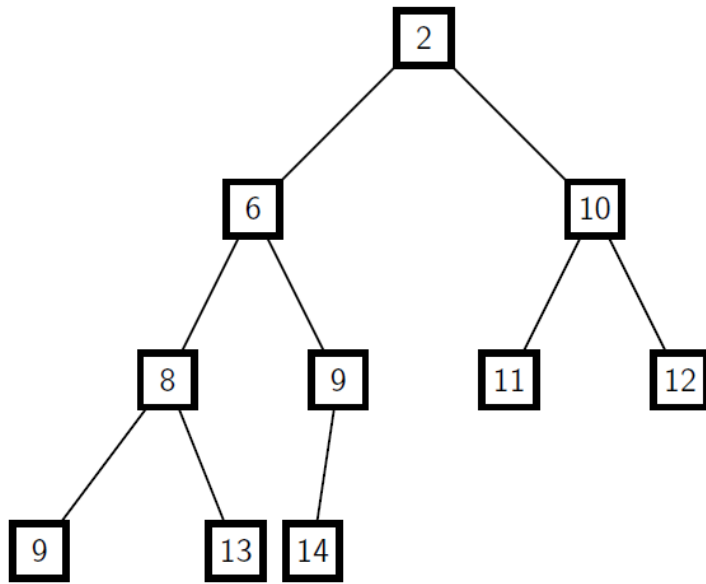
Un arbre binaire est dit **parfait** ssi tous ses niveaux sont complets sauf éventuellement le dernier et dans ce cas les feuilles sont groupées le plus à gauche possible.

Définition:

Un **tas** est un tableau représentant un arbre binaire parfait partiellement ordonné.

Chapitre2: Tas (Heap)

Exemple



$T =$

1	2	3	4	5	6	7	8	9	10
2	6	10	8	9	11	12	9	13	14

Propriétés de T

On suppose l'existence de la case $T[n + 1] = +\infty$

Théorème:

Soit n le nombre de nœuds du tas T .

□ Pour tout $i = 1, 2, \dots, \lfloor \frac{n}{2} \rfloor$ on a :

- ❖ le fils gauche du nœud d'indice i est le nœud $T[2i]$,
- ❖ le fils droit du nœud d'indice i est le nœud $T[2i + 1]$,

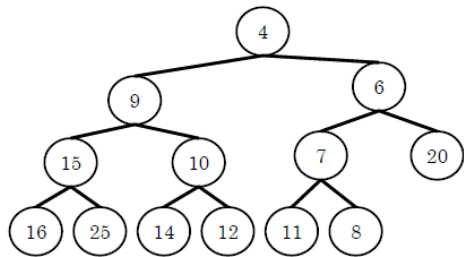
□ Pour tout $i = 2, 3, \dots, n$ on a :

- ❖ le père du nœud d'indice i est le nœud $T[\lfloor \frac{i}{2} \rfloor]$,

Chapitre2: Tas (Heap)

Ajout de nœuds dans un Tas

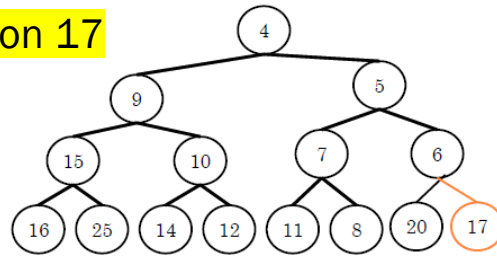
Exemple : insertion des nœuds : 5 , 17 , 3 , 18



T=

1	2	3	4	5	6	7	8	9	10	11	12	13
4	9	6	15	10	7	20	16	25	14	12	11	8

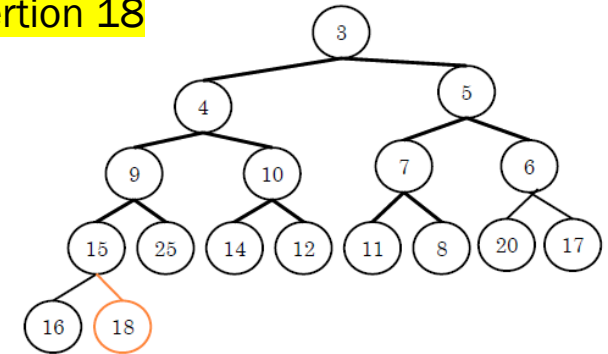
Insertion 17



T=

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
4	9	5	15	10	7	6	16	25	14	12	11	8	20	17

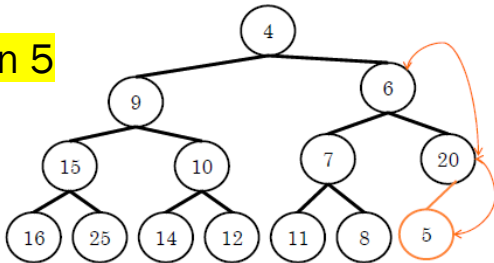
Insertion 18



T=

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
3	4	5	9	10	7	6	15	25	14	12	11	8	20	17	16	18

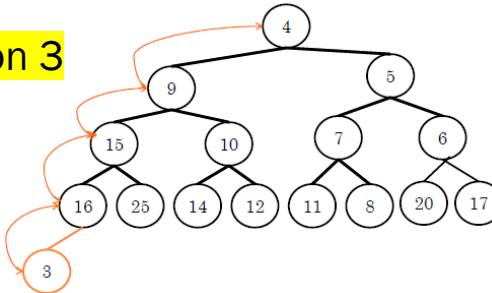
Insertion 5



T=

1	2	3	4	5	6	7	8	9	10	11	12	13	14
4	9	6	15	10	7	20	16	25	14	12	11	8	5

Insertion 3



T=

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16
4	9	5	15	10	7	6	16	25	14	12	11	8	20	17	3

Ajout de nœuds dans un Tas

```
Algorithme: insertion(x,T,n)
N = n +1 ;
T[n] = x ;
i = n;
tantque (i ≥ 1) et (T[i] < T[⌊ $\frac{i}{2}$ ⌋]) faire
    echanger (T[i] , T[⌊ $\frac{i}{2}$ ⌋]);
    i = ⌊ $\frac{i}{2}$ ⌋ ;
Fin
```

Nombre d'échanges = au pire la hauteur du tas.

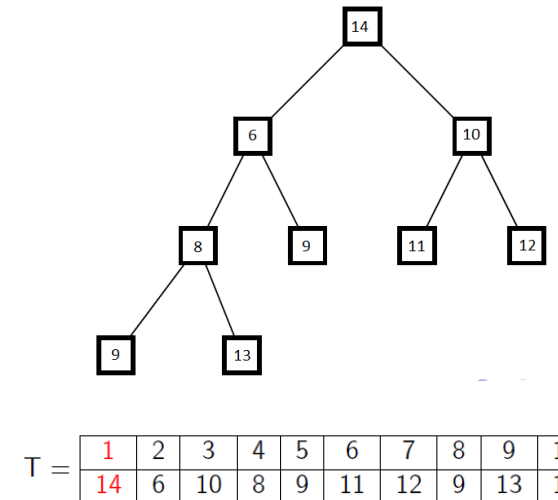
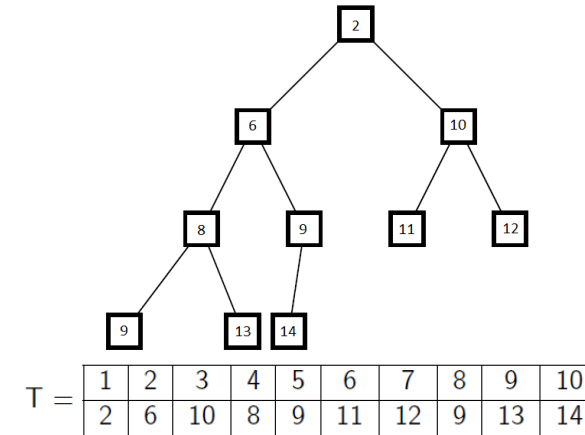
Chapitre2: Tas (Heap)

Suppression dans un Tas

Principe de suppression de la racine dans un tas:

1. Soit r le nœud racine et $\text{val}(r)$ sa valeur (1ère case du tableau)
2. $\text{Min} = \text{val}(r)$
3. Remplacer $\text{val}(r)$ par la dernière valeur du tas (la dernière case du tableau)
4. $m = \text{argmin}\{\text{val}(\text{gauche}(r)), \text{val}(\text{droite}(r))\}$
5. Tant que ($\text{val}(r) > \text{val}(m)$)
6. Echanger dans l'arbre les valeurs des nœuds r et m
7. $r = m$
8. $m = \text{argmin}\{\text{val}(\text{gauche}(r)), \text{val}(\text{droite}(r))\}$
9. Fin Tant que
10. Renvoyer Min

On appellera « Réorganisation » les instructions 4 à 9.



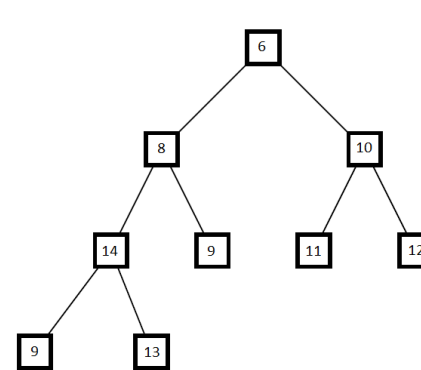
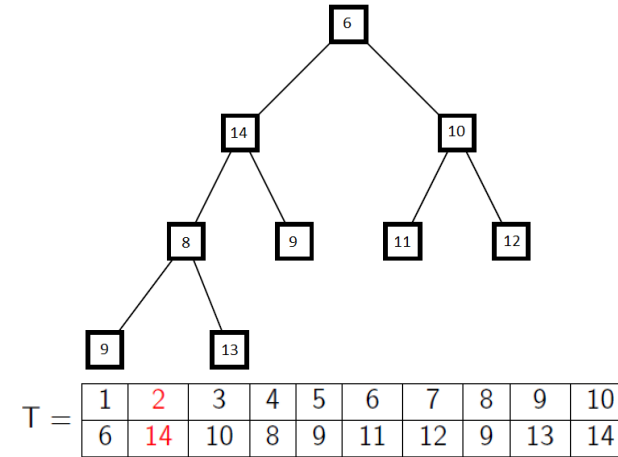
Chapitre2: Tas (Heap)

Suppression dans un Tas

Principe de suppression de la racine dans un tas:

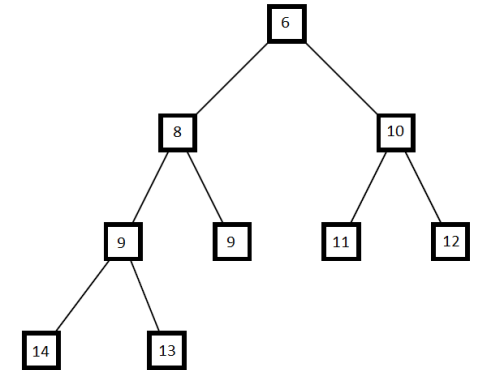
1. Soit r le nœud racine et $\text{val}(r)$ sa valeur (1ère case du tableau)
2. $\text{Min} = \text{val}(r)$
3. Remplacer $\text{val}(r)$ par la dernière valeur du tas (la dernière case du tableau)
4. $m = \text{argmin}\{\text{val}(\text{gauche}(r)), \text{val}(\text{droite}(r))\}$
5. Tant que ($\text{val}(r) > \text{val}(m)$)
6. Echanger dans l'arbre les valeurs des nœuds r et m
7. $r = m$
8. $m = \text{argmin}\{\text{val}(\text{gauche}(r)), \text{val}(\text{droite}(r))\}$
9. Fin Tant que
10. Renvoyer Min

On appellera « Réorganisation » les instructions 4 à 9.



T =

1	2	3	4	5	6	7	8	9	10
6	8	10	14	9	11	12	9	13	14



T =

1	2	3	4	5	6	7	8	9	10
6	8	10	9	9	11	12	14	13	14

Suppression dans un Tas

La réorganisation peut se faire à partir de n'importe quel indice j de T comme suit.

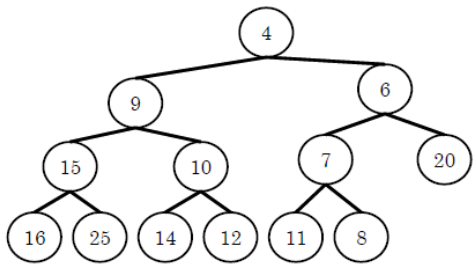
```
Algorithme: Reorganiser(T,j,n)
i = j ;
tant que (i ≤ ⌊ $\frac{n}{2}$ ⌋) faire
    Si (T[2i] < T[2i + 1]) alors
        m = 2i ;
    Fin
    Sinon
        m = 2i + 1 ;
    Fin
    Si (T[i] > T[m]) alors
        echanger ( T[i] , T[m]) ;
        i = m ;
    Fin
    Sinon
        i = n ;
    Fin
Fin
```

Nombre d'échanges = au pire la hauteur du tas.

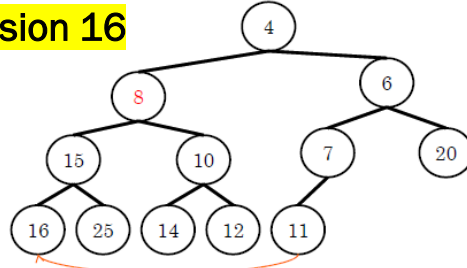
Chapitre2: Tas (Heap)

Suppression dans un Tas

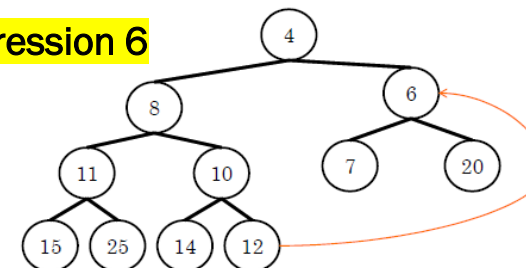
Exemple : suppression des nœuds : 9 , 16 et 6



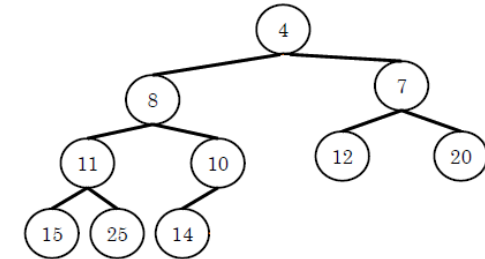
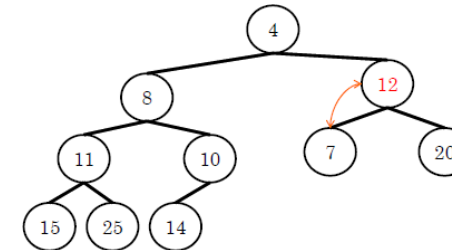
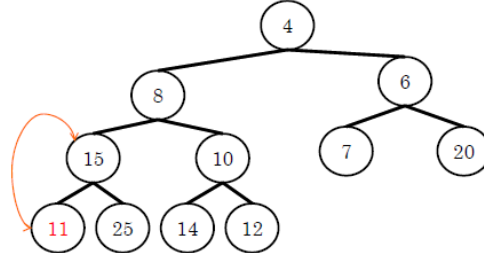
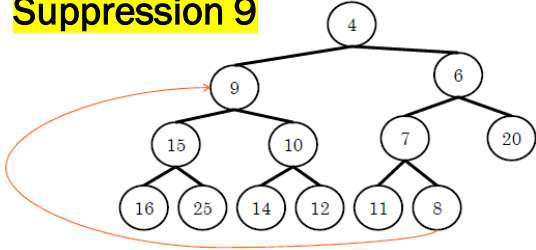
Suppression 16



Suppression 6



Suppression 9



Suppression dans un Tas

Algorithme de suppression

Algorithme: `Suppression(T,n)`

```
min  = T[1] ;
```

```
T[1] = T[n] ;
```

```
n = n-1 ;
```

```
Reorganiser(T , 1 , n);
```

```
Renvoyer min;
```

Tri par Tas

Si T est déjà un tas , le tri s'effectue comme suit:

```
Algorithme: TriTas(T,n)
k = n ;
tant que (k > 1) faire
    min = Suppression (T,k);
    k = k - 1 ;
    T[k+1] = min;
fin
```

Chapitre2: Tas (Heap)

Si T n'est pas un tas , il faut au préalable le réorganiser pour qu'il le devienne. Donc:

```
Algorithme: TriTas(T,n)
pour (k =  $\lfloor \frac{n}{2} \rfloor$  .. 1) faire
    Reorganiser(T,k,n);
fin
k = n ;
tant que (k > 1) faire
    min = Suppression (T,k);
    k = k - 1 ;
    T[k+1] = min;
fin
```

Hauteur d'un arbre binaire parfait

Chapitre2: Tas (Heap)

Théorème:

La hauteur d'un arbre binaire parfait de n nœuds est égale à $\lfloor \log_2(n) \rfloor$

Hauteur d'un arbre binaire parfait

Chapitre2: Tas (Heap)

Théorème:

La hauteur d'un arbre binaire parfait de n nœuds est égale à $\lfloor \log_2(n) \rfloor$

Preuve:

- Soit h , la hauteur d'un tas de n nœuds
- Au niveau $i \neq h$, $n_i = 2^i$
- Donc $n = 2^0 + 2^1 + 2^2 + \dots + 2^{h-1} + c$ tel que , $0 < c < 2^h$

$$n = 2^h + (c - 1) \geq 2^h \rightarrow h \leq \log_2(n) \dots \dots (1)$$

- Or: $n < 2^{h+1} \rightarrow \log_2(n) < h+1 \dots \dots \dots (2)$

- (1) et (2) donne: $h \leq \log_2(n) < h+1$

$$\text{Donc } h = \lfloor \log_2(n) \rfloor$$